Url

https://huggingface.co/datasets/Lichess/standard-chess-games

Descripcion

Lichess Standard Chess Games contiene partidas estándar con rating jugadas en Lichess. El dataset se actualiza y su ficha muestra cifras distintas según la revisión consultada, por lo que conviene registrar la fecha o revisión descargada y verificar su tamaño a la hora de usar el dataset.

Los datos están en formato Parquet, particionados por año y mes. Cada fila representa una partida e incluye, entre otros, estos campos:

- Resultado: victoria de blancas, victoria de negras o tablas.
- Ratings: Elo de ambos jugadores y cambio de rating tras la partida.
- Fecha y hora: fecha y hora UTC.
- Apertura: código ECO y nombre de la apertura.
- Control de tiempo: por ejemplo, bullet, blitz o classical.
- Finalización: por ejemplo, mate, abandono o caída de tiempo.
- Movimientos: texto de la partida en formato PGN (SAN).
- Jugadores y evento: nombres de usuario y títulos, cuando están disponibles.

Peso

4.94 TB

Objetivos

Objetivo	**Tipo de Problema**	**¿Que se predice o analiza?**	**Variables Posibles**	**Modelos / Metodos**	**Metricas Sugeridas**
Predecir el resultado de una partida con el Opening	Clasificación multiclase	Victoria de blancas, tablas o victoria de negras	Diferencia de rating, apertura, control de tiempo, títulos de jugadores, año y mes.	Random Forest, XGBoost, NN	Macro-F1, balanced accuracy, matriz de confusión
Predecir el resultado usando los primeros N	Clasificación multiclase	Resultado final usando, por ejemplo, los primeros 5, 10	Secuencia inicial de movimientos, apertura,	Regresión logística o Gradient Boosting con	Macro-F1, balanced accuracy, matriz de

Objetivo	**Tipo de Problema**	**¿Que se predice o analiza?**	**Variables Posibles**	**Modelos / Metodos**	**Metricas Sugeridas**
movimientos		o 15 movimientos	ratings y control de tiempo	características derivadas; modelos de secuencias como extensión	confusión
Estimar la duración de una partida	Regresión o clasificació	Cantidad total de movimientos o una categoría de duración: corta, media o larga	Apertura, diferencia de rating, control de tiempo, resultado y tipo de finalización	Regresión lineal, Random Forest, Gradient Boosting	MAE y RMSE para regresión; macro-F1 para clasificación
Predecir el cambio de rating de un jugador	Regresión o clasificación	Cambio de rating, o si el jugador ganará, perderá o mantendrá rating	Rating inicial, diferencia de ratings, resultado, apertura y control de tiempo	Regresión lineal, Random Forest, Gradient Boosting	MAE y RMSE para regresión; macro-F1 para clasificación
Agrupar aperturas o partidas por patrones similares	Clustering / análisis exploratorio	Grupos de aperturas o perfiles de partidas con características parecidas	Frecuencia de apertura, resultados, ratings, duración y modalidad	K-means, clustering jerárquico, PCA o UMAP para explorar	Métricas de clustering, como silhouette; interpretación de los grupos

Arquitectura IA

FT-Transformer (Feature Tokenizer + Transformer)

Solución

- main.py coordina la ejecución y la configuración.
- Módulos separados para ingesta, limpieza, preprocesamiento, muestreo, FT-Transformer y evaluación.
- El pipeline recorre y prepara los 2 TB por particiones; el modelo se entrena con una muestra reproducible del conjunto de entrenamiento.
- Result es la etiqueta. Se excluyen movetext, Termination, cambios de rating e identificadores de jugadores.

- Se evalúa con separación cronológica, macro-F1, balanced accuracy y matriz de confusión.

Pipeline offline, orquestado por main.py y dividido en módulos importables. Clasificando el resultado final usando como etiqueta Result después de identificar Opening, usando además como variables disponibles como los ratings de ambos jugadores, la diferencia y el promedio de ratings, sus títulos, el control de tiempo, openings, y la fecha; por ejemplo, año y mes. Se deben excluir las variables de cambios de rating y la terminación, que solo se conocen después de la partida; también se excluyen movetext y los nombres de usuario, por ser secuencias posteriores o identificadores con riesgo de fuga de información.

El flujo se resume en:
Leer los Parquet por particiones → limpiar y validar todos los registros → crear variables disponibles al momento de la predicción, una vez identificada la apertura → separar entrenamiento, validación y prueba cronológicamente → tomar la muestra sólo del conjunto de entrenamiento → ajustar el preprocesamiento con entrenamiento → entrenar y evaluar FT-Transformer → guardar modelo, preprocesador, métricas y configuración de la ejecución.

Se comparan los resultados del FT-Transformer con un modelo de XGBoost y una NN. Se medirían macro-F1, balanced accuracy y matriz de confusión, atendiendo al posible desbalance de las tablas.

Se recorren y preparan todos los datos, pero se entrena con una muestra representativa del conjunto de entrenamiento debido a que con miles de millones de partidas, el entrenamiento completo puede requerir recursos y tiempo muy altos.

Estructura de Archivos

main.py
src/lichess_pipeline/
  config.py
  data_ingestion.py
  data_cleaning.py
  data_preprocessing.py
  sampling.py
  model.py
  train.py
  evaluate.py

Diagrama C4